

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ
«МИСИС»**

ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИKN)

Курсовая работа на тему:

«Алгоритм Левита, построение кратчайших расстояний»

Выполнила: студентка группы

БИВТ-23-5 Крюкова Екатерина

Николаевна

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
ОСНОВНЫЕ ТЕРМИНЫ	4
Сравнительный анализ	5
Шаги реализации алгоритма Левита.....	6
Анализ алгоритма	6
Реализация	8
ЗАКЛЮЧЕНИЕ.....	15
ИСТОЧНИКИ.....	16

ВВЕДЕНИЕ

Алгоритм Левита - это метод нахождения кратчайших путей в графе от одной начальной вершины до всех остальных. Он работает с графами, содержащими рёбра как с положительными, так и с отрицательными весами, при условии отсутствия отрицательных циклов. Алгоритм использует три множества вершин для эффективного обновления расстояний: вершины, готовые к обработке, вершины, требующие повторного рассмотрения, и вершины с уже найденными кратчайшими путями.

В процессе работы алгоритм перемещает вершины между этими множествами в зависимости от их текущего состояния. Это позволяет эффективно находить оптимальные пути, особенно в графах с отрицательными весами рёбер, где классический алгоритм Дейкстры не может быть применим.

Алгоритм применяется для оптимизации маршрутов, моделирования сетей и других задач, связанных с анализом графов.

Псевдокод:

Для хранения вершин используются следующие структуры данных:

M0 — хеш-таблица,

M1 — основная и срочная очереди,

M2 — хеш-таблица.

```

for  $u : u \in V$ 
     $d[u] = \infty$ 
 $d[s] = 0$ 
 $M_1'.$ push( $s$ )
for  $u : u \neq s$  and  $u \in V$ 
     $M_2.$ add( $u$ )
while  $M_1' \neq \emptyset$  and  $M_1'' \neq \emptyset$ 
     $u = (M_1'' = \emptyset ? M_1'.pop() : M_1''.pop())$ 
    for  $v : uv \in E$ 
        if  $v \in M_2$ 
             $M_1'.$ push( $v$ )
             $M_2.$ remove( $v$ )
             $d[v] = \min(d[v], d[u] + w_{uv})$ 
        else if  $v \in M_1$ 
             $d[v] = \min(d[v], d[u] + w_{uv})$ 
        else if  $v \in M_0$  and  $d[v] > d[u] + w_{uv}$ 
             $M_1''.$ push( $v$ )
             $M_0.$ remove( $v$ )
             $d[v] = d[u] + w_{uv}$ 
     $M_0.$ add( $u$ )

```

Рис.1 – Псевдокод.

ОСНОВНЫЕ ТЕРМИНЫ

Перед тем как познакомиться с этим алгоритмом подробнее необходимо ознакомиться со следующими терминами:

1. Вес ребра

Числовое значение, ассоциированное с ребром. Может быть положительным или отрицательным (если нет отрицательных циклов).

2. Кратчайший путь

Оптимальный маршрут между двумя вершинами, имеющий минимально возможную суммарную стоимость (наименьшую сумму весов всех входящих в него рёбер).

3. Релаксация

Это процесс, который позволяет постепенно уточнять оценки расстояний до вершин в алгоритмах поиска кратчайших путей. Именно она является главной частью алгоритма.

4. Множества вершин в алгоритме Левита

В процессе работы алгоритм разделяет вершины на три категории:

- Необработанные - вершины, до которых пути ещё не найдены.
- Текущие - вершины, находящиеся в обработке.
- Обработанные - вершины с окончательно определёнными кратчайшими расстояниями.

5. Отрицательный цикл

Цикл, сумма весов рёбер которого отрицательна. Если граф содержит такой цикл, алгоритм сигнализирует, что корректный кратчайший путь невозможно определить.

6. Очереди обработки вершин

- Алгоритм использует две специализированные структуры:
 - Основная очередь (FIFO) - для последовательной обработки вершин текущего уровня.
 - Дек (двусторонняя очередь) - для приоритетного анализа соседних вершин и оптимизации вычислений.

7. Вершина-предшественник

Вершина, через которую был найден кратчайший путь.

8. Граф

Это математическая структура, которая используется для описания множества объектов и связей между ними.

Сравнительный анализ

Алгоритм Левита предназначен для нахождения кратчайших путей от исходной вершины ко всем остальным в графе, как и классические алгоритмы Дейкстры и Беллмана-Форда. Однако эти методы различны по своим характеристикам и областям применения.

Сравнение

Критерий	Левит	Дейкстра	Беллман-Форд
Графы	Работает с любыми графами, кроме содержащих отрицательные циклы.	Только для графов с положительными весами.	Работает со всеми типами графов, обнаруживает отрицательные циклы.
Сложность	$O(V+E)$ для разреженных графов (с использованием дека).	$O((V+E) \cdot \log V)$ с приоритетной очередью.	$O(V \cdot E)$ - самая высокая среди трех.
Производительность	Оптимален для разреженных графов с отрицательными весами.	Самый быстрый для графов с положительными весами.	Универсальный, но самый медленный.
Обработка отрицательных весов	Полная поддержка (исключая циклы).	Не поддерживается.	Полная поддержка с обнаружением циклов.
Используемые структуры	Комбинация дека и очереди.	Приоритетная очередь.	Простейший список рёбер.
Применение	Транспортные системы с переменными тарифами.	GPS-навигация, сетевые протоколы.	Финансовый анализ, моделирование сетей.
Преимущества	Работает с отрицательными весами. Быстрее Беллмана на разреженных графах.	Самая высокая скорость для положительных весов. Простая реализация.	Обнаруживает отрицательные циклы. Самая простая реализация.
Недостатки	Сложная реализация. Не находит отрицательные циклы.	Не работает с отрицательными весами.	Самая низкая скорость ($O(V \cdot E)$). Неэффективен для больших графов.

Шаги реализации алгоритма Левита:

1. На вход подается:
 - Граф (список смежности или матрица)
 - Стартовая вершина S
2. Инициализация:
 - Установим вершине S расстояние 0
 - Для всех остальных вершин устанавливаем расстояние бесконечность
 - Поместим вершину S в очередь
3. Основной цикл обработки
Пока очередь не пуста:
 - Извлекаем текущую вершину K из очереди
 - Анализируем всех соседей вершины K:
 - Для каждого соседа N вычисляем новое расстояние через вершину K
 - Если новое расстояние короче текущего:
 - Обновляем расстояние до N
 - Если N ещё не обрабатывалась, то добавляйте в конец очереди
 - Если N уже была в обработке, то оставляйте без изменений
 - Если N была обработана ранее, то поместите в начало очереди для повторного анализа
4. Завершение работы
 - Когда очередь станет пустой, кратчайшие расстояния до всех достижимых вершин будут определены.
 - Также можно восстановить конкретные маршруты, используя дополнительный массив предшественников

Анализ алгоритма

Корректность алгоритма:

Алгоритм работает правильно благодаря последовательному уточнению расстояний.

1. Начальное состояние:

- В самом начале только у стартовой вершины S расстояние равно 0
- У всех остальных вершин расстояние считается бесконечным

2. Предположение индукции:

- Предположим, что мы уже правильно нашли кратчайшие пути для первых h вершин. Теперь нужно доказать для $h+1$ вершины:

3. Обработка новой вершины:

- Когда мы достаём вершину из очереди, у неё уже минимальное возможное расстояние, так как все возможные пути к ней мы уже проверили через другие вершины
- Берём ближайшую доступную вершину.

4. Финальный вывод:

- Каждую вершину мы добавляем в завершённое множество только когда уверены в её расстоянии
- Значит, когда алгоритм заканчивает работу, все расстояния точно минимальные

Оценка временной сложности:

Обозначения:

- V — количество вершин,
- E — количество рёбер.

Основные компоненты сложности:

1. Инициализация структур данных:

- Заполнение массива расстояний: $O(V)$
- Подготовка очереди: $O(1)$

2. Обработка вершин:

- Каждая вершина добавляется в очередь максимум 2 раза
- Общее число операций с очередями: $O(V)$

3. Релаксация рёбер:

- Каждое ребро проверяется при обработке инцидентной вершины
- Всего выполняется $O(E)$ проверок рёбер

Общая временная сложность алгоритма:

$O(V+E)$, что делает его особенно эффективным на разреженных графах.

Оценка используемой памяти:

- Хранение графа: $O(E)$
- Массивы расстояний и меток: $O(V)$
- Структуры очередей: $O(V)$

Итоговая пространственная сложность: $O(V+E)$

Реализация

Рассмотрим реализацию данного алгоритма с пошаговым объяснением каждого шага и его визуализацией и сравним итоговый результат, который был получен при ручном анализе.

Для визуализации были использованы библиотеки `networkx` и `matplotlib`.

- Стартовая вершина S
 - Исходные расстояния:
 - S: 0
 - X: ∞
 - Y: ∞
 - Z: ∞
 - K: ∞
 - Начальная очередь: `ds[S]`
1. Новые расстояния:
- X: новое расстояние $0+4=4$, `ds(X)= 4`, предшественник `X=S`
 - Y: новое расстояние $0+1=1$, `ds(Y)= 1`, предшественник `Y=S`
 - Добавляем X и Y в очередь
2. Обработка вершины Y:
- Извлекаем Y из очереди (так как приоритет у вершины с меньшим расстоянием)
 - Обновляем расстояние:
 - X: $1+1=2$, `ds(X)=2`, предшественник `X=Y`
 - Z: новое расстояние $1+5=6$, `ds(Z)= 6`, предшественник `Z=X`

- Добавляем вершину Z в очередь.

3. Обработка вершины X :

- Извлекаем X из очереди
- Обновляем расстояние:
 - Z : новое расстояние $2+2=4$, $ds(Z)=4$, предшественник $Z=S$
 - Добавляем вершину Z в очередь.

4. Обработка вершины Z :

- Извлекаем Z из очереди
- Обновляем расстояние:
 - K : $4+3=7$, $ds(K)=7$, предшественник $K=Z$
 - Добавляем вершину K в очередь.

5. Обработка вершины K :

- Извлекаем K из очереди
- Обновляем K расстояние:
 - K нет соседей – конец.

Пошаговая визуализация алгоритма:

1. Начальное состояние.

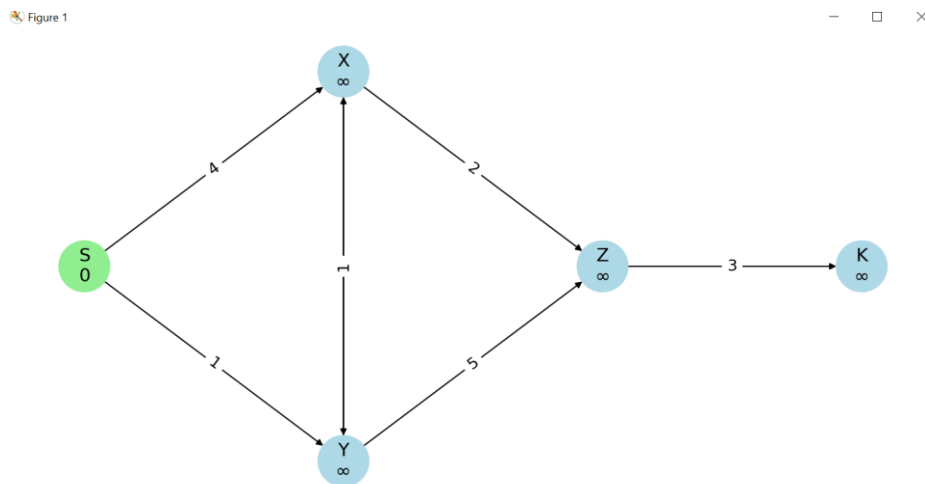


Рис.2 Начальное состояние.

2. Шаг 1:

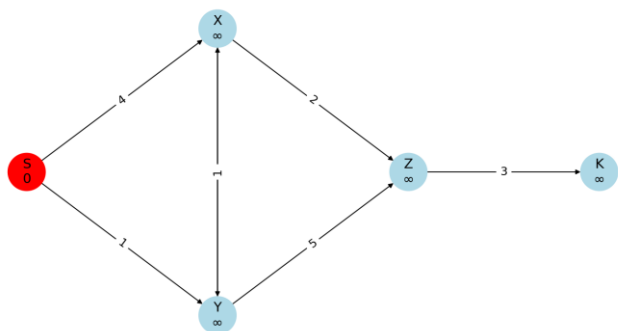


Рис.3 Шаг 2.

3. Шаг 2:

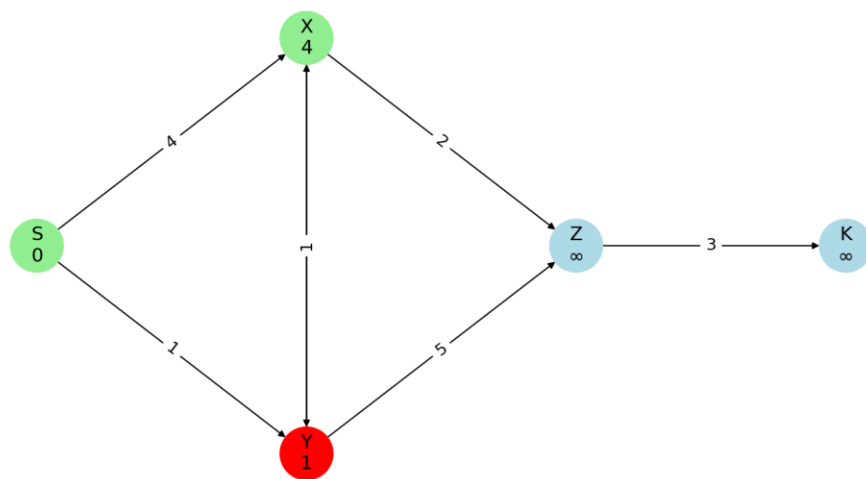


Рис.4 Шаг 2.

4. Шаг 3:

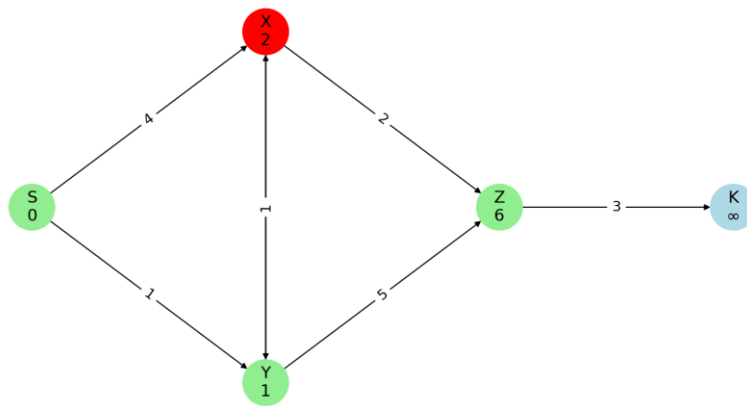


Рис.5 Шаг 3.

5. Шаг 4:

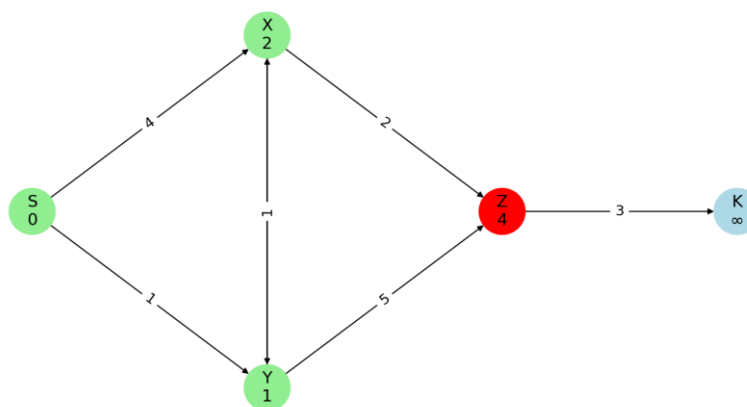


Рис.6 Шаг 4.

6. Шаг 5:

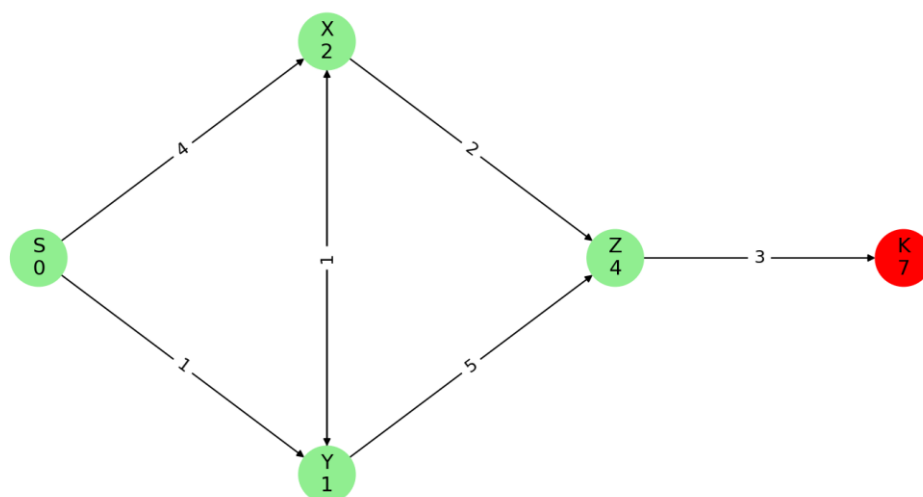


Рис.7 Шаг 5.

Итог:

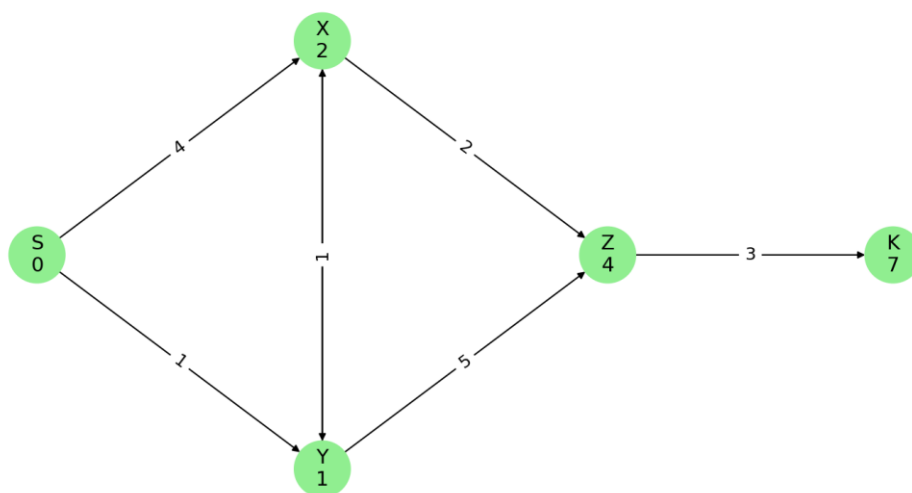


Рис.8 Итог.

Напишем реализацию на питоне на основе предыдущего

примера:

```
import heapq
def dijkstra(graf, spisok_vershin, start):
    rasstoyaniya = {v: float('inf') for v in spisok_vershin}
    rasstoyaniya[start] = 0
    ochered = [(0, spisok_vershin.index(start))]

    while ochered:
        tekush_rasst, tekush_ind = heapq.heappop(ochered)
        tekush_versh = spisok_vershin[tekush_ind]
        if tekush_rasst > rasstoyaniya[tekush_versh]:
            continue
        for sosed, ves_rebra in graf[tekush_ind]:
            novoe_rasst = tekush_rasst + ves_rebra
            if novoe_rasst < rasstoyaniya[sosed]:
                rasstoyaniya[sosed] = novoe_rasst
                heapq.heappush(ochered, (novoe_rasst, spisok_vershin.index(sosed)))

    return rasstoyaniya
graf = [
    [('X', 4), ('Y', 1)], #S
    [('Y', 1), ('Z', 2)], #X
    [('X', 1), ('Z', 5)], #Y
    [('K', 3)], #Z
    [], #K
]
vershiny = ['S', 'X', 'Y', 'Z', 'K']
startovaya = 'S'
itog = dijkstra(graf, vershiny, startovaya)
print("Кратчайшие расстояния от S:")
for vershina, rast in itog.items():
    print(f"{vershina}: {rast if rast != float('inf') else '∞'}")
```

Рис.9 Код.

Вывод:

```
Кратчайшие расстояния от S:
S: 0
X: 2
Y: 1
Z: 4
K: 7
```

Рис.10 Результат.

Проанализируем этот алгоритм вручную:

Прямой путь от S к Z через вершину X даст нам расстояние 6, но через вершину Y составляет 4, что намного выгоднее. Также расстояние до X через Y выгоднее, чем через прямое ребро. Для вершины K единственный маршрут $S - Y - X - Z - K = 7$. Таким образом мы можем сделать вывод, что алгоритм работает верно и на каждом шаге действительно выбирается минимальное расстояние.

ЗАКЛЮЧЕНИЕ

Алгоритм Левита эффективно находит кратчайшие пути в графах с положительными весами, используя оптимизированную обработку вершин через двустороннюю очередь. Он превосходит алгоритм Дейкстры на плотных графах благодаря динамическому управлению порядком вычислений. Особенно полезен для навигации, маршрутизации и логистики, где важна скорость. Хотя реализация сложнее классических методов, его производительность оправдывает применение в задачах с большими графами.

ИСТОЧНИКИ

1. https://руни.рф/Алгоритм_Левита
2. https://nerc.itmo.ru/wiki/index.php?title=Алгоритм_Левита
3. <https://leventov.livejournal.com/14855.html>
4. <https://github.com/ekaterina978/molnuiyamakvin>